Wilhelm Büchner University of Applied Sciences

Projektarbeit Master

Thema:
Konzeptionieren eines Coasterbots

Endbericht



Team: Coasterbot

Studierender	**Matrikelnummer**
Dennis Borutta, B. Eng	857962
Stefan Etringer, B. Eng	865769
Gereon Such, M. Sc.	867070

Betreuer: Dr.-Ing. Winfried Tiedge

Abgabedatum: 07.11.2026

Versionshistorie

Version	Datum	Bearbeiter	Änderungen
1	25.09.2026	Borutta	Erster Entwurf.
2	08.10.2026	Such	Erstfassung eigener Anteile.
3	10.10.2026	Etringer	Beschreibung der Algorithmik und Simulation.
4	10.10.2026	Etringer	Beschreibung des Ergebnis und des Nutzens.
5	15.10.2026	Borutta	Beschreibung Projektziel und der Mechanik.

Inhaltsverzeichnis

1 Projektumsetzung **4**

1.1 Projektziel (Dennis Borutta) 4
1.2 Lösungsweg: Mechanischer Aufbau (Dennis Borutta) 5
1.3 Lösungsweg: Algorithmik und Simulation (Stefan Etringer) 6
1.4 Lösungsweg: Elektronik und Abstraktion (Gereon Such) 8
1.5 Projektergebnis (Dennis Borutta) 13
1.6 Zeitplan Soll/Ist (Dennis Borutta) 13
1.7 Kosten- und Ressourcenplan Soll/Ist (Gereon Such) 13
1.8 Nutzen und Weiterentwicklung des Ergebnis (Stefan Etringer) 13

2 Projektablauf **15**

2.1 Teambildung und Rollenverteilung (Dennis Borutta) 15
2.2 Zusammenarbeit im Projekt (Stefan Etringer) 15
2.3 Kommunikation im Projekt (Gereon Such) 16
2.4 Bewertung der Zusammenarbeit 18
2.5 Bewertung des Projektergebnis 18

1 Projektumsetzung

Planumfang ca. 9 Seiten Davon: Dennis 3 Seiten, Stefan 3 Seiten, Gereon 3 Seiten

1.1 Projektziel (Dennis Borutta)

Inhalt ca 0,5 Seiten

Mit diesem Projekt liefert das fiktive Ingenieurbüro „B · E · Sondere Lösung UG“ für den fiktiven Kunden „Pay With Fun GmbH“ einen Prototypen-Konzept für einen Coasterbot. Dabei ist das explizite Ziel Grundfunktionen in dem Prototypen umzusetzen. Diese Grundfunktionen umfassen folgendes:

- Navigation auf einem Tischoberfläche
- Ausgabe von Getränkeuntersetzer
- Lokalisierung von Getränkeuntersetzer
- Aufnahme von Getränkeuntersetzer

Zur Umsetzung dieser Grundfunktionen dürfen bereitsbestehende Konzepte zu mobilen Robotern verwendet werden. Innerhalb des Projektes sind weitere Ausbaustufen auszuarbeiten und mit einem Testkonzept zu versehen.

Zu diesem Prototypen sind folgende Liefergegenstände bereitzustellen:

- Eine Aufbauanleitung
- Eine Stückliste (Bill of Materials, BOM)
- Die zur Fertigung notwendigen Modelle als stl-Dateien
- Ein Simulationsmodell zur Weiterentwicklung
- Der Sourcecode zum Prototypen

Mit dem Upload dieser Liefergegenstände über das Dokumenten-Management-System (DMS) der Wilhelm-Büchner-Hochschule (WBH) gelten sie als ausgeliefert. Ein physischer Aufbau wird nicht separat geliefert werden.

Nicht als Bestandteil des Projekts gelten:

- Ein Serienfertiges Endprodukt
- EMV Betrachtungen
- Intelligentes Schwarmverhalten
- Funktionen außerhalb der Grundfunktionen
- Konzeption einer Vermarktung des Produkts

Damit wird das Projekt als „PA2 - Machbarkeitsprojekt“ im Maßstab der Wilhelm-Büchner-Hochschule geführt. Der Projekstart erfolgte am 21.07.2026 und das Projektende ist am 07.11.2026 mit der Abgabe dieses Endbericht. Damit beträgt der angesetzte zeitliche Rahmen 3,5 Monate angesetzt.

1.2 Lösungsweg: Mechanischer Aufbau (Dennis Borutta)

Inhalt ca 1 Seite

- Erarbeitung des Lösungsweges: Benchmarking, Abgrenzung Ihres Vorgehens und der Wahl der Mittel von alternativen Lösungsmöglichkeiten, Begründung Ihres Ansatzes

Ausgangslage und Leitlinien

Zur Umsetzung der Mechanik sind folgende Anforderungen aus dem Pflichtenheft als Leitlinien herangezogen:

- (HW-ACT-005..007) Anforderungen an das Getränkeuntersetzerhandling.
- (HW-MEC-001..005) Anforderungen an die mechanische Konstruktion.

Benchmarking und Auswahl der Verfahren

Zur Auswahl der Lösungmethoden wurden unterschiedlichen Lösungsoptionen recherchiert und Gegenüberstellungen. Dies ist ausführlich im Dokument *alternativenbewertung* beschrieben und im DMS der WBH, daher werden an dieser Stelle lediglich gewählten Lösungswege mit ihren Begründungen verkürzt vorgestellt.

Tabelle 1: Betrachtete Alternativen und gewählte Lösung der Mechanik.

Funktion	Alternativen	Gewählt	Begründung
Chassis - Material	Holz, Metall	Kunststoff (Polylactid)	Kunststoff ist günstig und Polyactid lässt sich besonders einfach auf einem 3D-Drucker verarbeiten.
Chassis - Form	Abgerundet	Rechteckig	Durch die rechteckige Form sind nachträgliche Anpassungen einfach anzubringen und die Konstruktion einfacher.
Chassis - Modularität	Zwischenebenen	Austauschbare Seitenwände	Druck kann in einzelnen Teilen erfolgen und die Wände sind austauschbar (Es muss kein gesamtes neues Gehäuse hergestellt werden).
Untersetzer ablegen	Roboter mit Greifer/Sauger	Ausschieber	Weniger komplex und benötigt weniger Antriebe.
Untersetzer aufnehmen	Roboter mit Greifer/Sauger, Manuelle Einlage	Magnetische Aufnahme	Weniger komplex und benötigt weniger Antriebe.

Anpassungen während der Umsetzung

1.3 Lösungsweg: Algorithmik und Simulation (Stefan Etringer)

Ausgangslage und Leitlinien

Das Pflichtenheft fordert vom Coasterbot eine zielgerichtete autonome Bewegung auf der Tischfläche (`NAV-*`), Hindernis- und Tischkantenerkennung sowie eine Sicherheitskomponente mit übergeordneter Priorität (`SAF-*`). Als Hardware wurde ein Mikrocontroller (Raspberry Pi Pico 2) statt eines SBC mit ROS gewählt. Deshalb wurde entschieden, die Navigation zunächst in einer Physiksimulation (Webots) zu entwerfen und gegen die bereits festgelegten Testfälle aus dem Testkonzept zu verifizieren. Dadurch können die gewonnenen Erkentnisse auf die reale Hardware übertragen werden. Entscheidend dafür ist die Hardwareabstraktionsschicht `RobotHAL` (siehe Abschnitt 1.4): Navigations- und Sicherheitslogik kennen ausschließlich dieses Interface und sollen dadurch unverändert von der Simulation auf die Zielhardware übernommen werden können.

Benchmarking und Auswahl der Verfahren

Grundlage war eine systematische Literaturrecherche (M0, *Algorithmen.md*) zu Graphalgorithmen, heuristischer und dynamischer Pfadplanung, lokaler Kollisionsvermeidung, Lokalisierung sowie Manipulationsplanung. Die daraus abgeleiteten Möglichkeiten und Alternativen wurden in einer Mindmap (M3, *KonzeptMindMap-Algorithmen.svg*) je

Teilproblem strukturiert und in der Alternativenbeschreibung (M3) anhand der festgelegten Anforderungen bewertet: begrenzte Tischfläche, begrenzte Rechenleistung sowie der sicherheitskritischen Anforderung, den Tisch nicht zu verlassen. Tabelle 2 fasst die resultierende Auswahl zusammen.

Tabelle 2: Betrachtete Alternativen und gewählte Lösung der Navigationsalgorithmik.

Funktion	**Alternativen**	**Gewählt**	**Begründung**
Globale Pfadplanung	Dijkstra, Breitensuche	A*	Heuristik lenkt die Suche zielgerichtet, bleibt auf dem Gitter dennoch optimal.
Lokale Kollisionsvermeidung	Potentialfelder, adaptive/neuronale DWA	Dynamic Window Approach	Bezieht Bewegungsdynamik ein, vermeidet lokale Minima der Potentialfeldmethode.
Dynamische Neuplanung	D*, LPA*	D* Lite	Etabliertes, inkrementelles Verfahren für nachträglich entdeckte Hindernisse.
Lokalisierung	IMU/INS allein, voller EKF/UKF, SLAM	Radodometrie + Gyro, Komplementärfilter	Für die bekannte, flache Tischfläche ausreichend, SLAM nicht erforderlich.
Manipulationsplanung	Inverse Kinematik + RRT*	nicht umgesetzt	Laut Abgrenzung nicht Teil der Simulationssoftware in M4.

Von der Recherche zur Simulation

Die gewählten Verfahren wurden als eigenständige, nur über `RobotHAL` angebundene C++-Module umgesetzt: Ein `OccupancyGrid` bildet die Tischfläche als Gitterkarte mit Distanzfeld ab, bekannte Hindernisse werden direkt eingetragen, live erkannte erst nach mehreren konsistenten Messungen. `astarPlan` liefert darauf einen geglätteten Pfad, `DStarLite` plant inkrementell um, sobald der Ultraschallsensor ein unkartiertes Hindernis entdeckt. Der `DwaPlanner` bestimmt daraus jeden Regelzyklus das Geschwindigkeitspaar beider Seiten. Die Lokalisierung liefert der `PoseEstimator` über einen Komplementärfilter aus Radodometrie und Gyroskop, der beim Schlupf des Skid-Steer-Antriebs in Kurven stärker auf das Gyroskop vertraut. Außerdem gibt es die Sicherheitsebene: `SafetyMonitor` reagiert mit übergeordneter Priorität auf Tischkanten, `FaultMonitor` zusätzlich auf Sensor- und Aktorausfälle. Dedizierte Betriebsarten erlaubten es also, jede Stufe einzeln und reproduzierbar zu prüfen, bevor sie Teil des Gesamtsystems wurde.

Anpassungen während der Umsetzung

Mehrere Entscheidungen wurden angepasst, nachdem sich in der Simulation Probleme verdichteten. Der volle Sensorfusions-Ansatz (EKF/UKF) wurde zugunsten des einfacheren Komplementärfilters zurückgestellt. Dies ist für die bekannte Tischfläche ausreichend

genau, allerdings ohne Unsicherheitsschätzung der Pose. Da die lokale Bewegungssteuerung dem kürzesten Pfad von A* nahe von Ecken nicht zuverlässig folgen konnte, wurden zwei unterschiedlich stark inflatierte Gitter (Planung/Kollisionsvermeidung) sowie eine Pfadglättung eingesetzt. Diese Pfadplanun verschiebt Wegpunkte von Ecken weiter zur Mitte. Eine einzelne Kostenkarte mit weicheren Gradienten würde diese Zweiteilung künftig überflüssig machen. Der `FaultMonitor` war in der ursprünglichen Mindmap noch nicht vorgesehen und kam erst hinzu, als sich die Testfälle zu Sensor- und Aktorausfällen konkretisiert haben. Bei seiner eigenen Verifikation fiel dabei eine zu eng gewählte Plausibilitätsschwelle auf, die normales Sensorrauschen fälschlich als Ausfall wertete. Dies wurde korrigiert. Bewusst nicht umgesetzt blieben die Manipulationsplanung sowie die missionsweite Ablaufsteuerung (`IDLE` $\rightarrow$ ... $\rightarrow$ `DONE`), für die inzwischen ein mit der Elektronik abgestimmter Vorschlag existiert.

In der simulierten Verifikation bestanden alle bearbeiteten Testfälle: Zielfehler rund 5.8 cm, Hindernisabstand mindestens 0.22 m, zuverlässige Tischkantenerkennung, byte-identische Wiederholläufe sowie Sensor-/Aktorfehlererkennung innerhalb von 0.02 s bzw. 0.34 s. Offen bleiben nur die ohnehin nicht im Scope liegenden Testfälle zum Untersetzerhandling sowie der Abgleich gegen den realen Prototyp.

1.4 Lösungsweg: Elektronik und Abstraktion (Gereon Such)

Inhalt ca 2 Seiten

- Erarbeitung des Lösungsweges: Benchmarking, Abgrenzung Ihres Vorgehens und der Wahl der Mittel von alternativen Lösungsmöglichkeiten, Begründung Ihres Ansatzes

Ausgangslage und Leitlinien

Die im Pflichtenheft definierten Hardwarefunktionen mussten durch entsprechende Sensorik, Aktorik und Rechenleistung abgedeckt werden:

- `HW-CPU-001..004` Rechenleistung für Sensordatenverarbeitung und Navigation
- `HW-SEN-001..005` Hindernis und Tischkantenerkennung
- `HW-SEN-006/007` Positionierung und Orientierung
- `HW-ACT-001..004` Bewegungssteuerung
- `HW-ACT-005..007` Untersetzeraufnahme und -ausgabe
- `HW-PWR-*` Energieversorgung

Ein weiteres Kriterium bei der Komponentenauswahl war schnelle und problemlose Verfügbarkeit in geringen Stückzahlen sowie ein geringer Preis. Da eine PCB Fertigung inklusive Versand einen enormen Zeitaufwand impliziert, z.B. JLCPCB ca. 4 Wochen, wurden alle Komponenten als Development-Boards beschafft und diese auf Lochrasterplatine sowie mit Steckverbindern auf Pinheadern verbunden. Jede Komponente wurde iterativ in Betrieb genommen und die Funktionalität sowie Eignung einzeln verifiziert. Dabei ergeben sich in einzelnen Fällen Schwierigkeiten, die Anpassungen erforderten, dies soll im Anschluss an die ursprüngliche Auswahl der Komponenten erläutert werden.

Benchmarking und Auswahl der Komponenten

Die Auswahl der Komponenten wurde in Phase M3 (Alternativenbeschreibung) diskutiert, hier wurden für jede Anforderung Lösungsvarianten identifziert und eine ausgewählt. Tabelle 3 fasst die wesentlichen Entscheidungen zusammen.

Tabelle 3: Betrachtete Alternativen und gewählte Lösung der Elektronik.

Funktion	**Alternativen**	**Gewählt**	**Begründung**
Controller	Arduino Uno, ESP32, Raspberry Pi Pico2	Raspberry Pi Pico 2 (RP2350)	Uno nicht performant genug. Pico2 nimmt weniger Leistung auf als ESP32 und hat mit PIO Interface Möglichkeit zeitkritische Signale zu generieren.
Hindernis-erkennung	Ultraschall HC-SR04, Kamera OV7670, Bumper, IR-Sensor	HC-SR04 in Fahrtrichtung	Unfälle auf dem Tisch sind zu wahrscheinlich mit Bumper Sensor. HC-SR04 bietet einfache und günstige Distanzmessung ohne Bildverarbeitung. Die Kamera wurde wegen des Verarbeitungsaufwands auf dem Mikrocontroller zurückgestellt.
Tischkanten-erkennung	IR-Reflexsensor, Kamera	4 IR-Sensormodule (je Ecke)	Kamera zurückgestellt, daher alternativlos.
Lage und Odometrie	IMU, Radencoder, Kamera	MPU-6050 (I^2C) und LM393-Gabellichtschranken	Die IMU (Inertialsystem) liefert die Drehung, die Encoder die gefahrene Strecke. Die Radencoder wurden während der Testphase ergänzt (siehe unten).
Untersetzer-mechanik	Micro-Servo, Schrittmotor	2 Micro-Servos (MG90)	Positionsgeregelt, nur ein Signalpin, aufgrund hohen Widerstands des PETG Aufbaus mussten Metallgewinde gewählt werden.
Antrieb	TT-Motoren mit H-Brücke	4 TT-Motoren (1:48), 2× TB6612FNG	Günstig und als Set mit Rädern verfügbar. Der Regler TB6612FNG wurde aufgrund geringerer Verlustleistung als der L298N gewählt.
Energie-versorgung	9-V-Block mit Linearregler 78x05; Li-Ion-Akku mit DC/DC-Wandler	18650-Li-Ion-Akku (3.7 V) mit Step-Up-Wandler auf 5 V	Die gewählten Motoren bedürfen recht hoher Ströme, dieser kann nicht durck Linearregler geliefert werden. Folglich Motorspannung `VM` direkt am Akku, Steuerspannung `VCC` über Step-Up-Wandler.
Bedienung und Status	RGB-LED oder diskrete LEDs; Taster	RGB-LED, Taster, Hauptschalter	Eine RGB-LED kann alle Betriebszustände über Farbe und Blinkmuster darstellen.

Elektrischer Aufbau

Die Schaltung wurde in der Open Source Software KiCad erstellt (Abbildung 1) und mit der Software DIYLC auf ein Lochrasterplatinenlayout im Raster 30×24 übertragen. Externe Komponenten werden über Pin Header angeschlossen. Die Platine trägt den Mikrocontroller, den Step-Up-Wandler, die Motortreiber, das Inertialsystem, die RGB-LED, Glättungskondensatoren für die Versorgungsspannungen sowie Pulldown Widerstand und Endprellung für den Taster. Der Aufbau ist in der Aufbauanleitung (M4) detailliert dokumentiert.

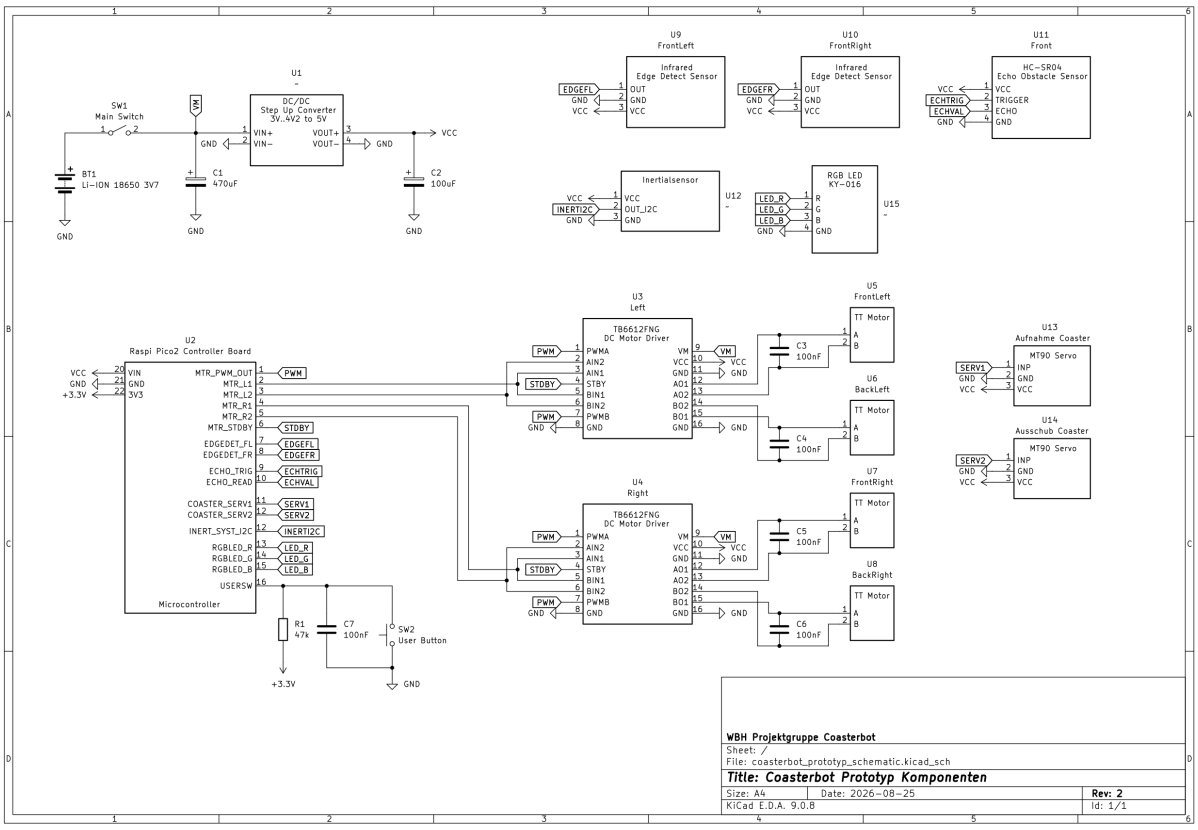


Abbildung 1: Gesamtschaltung der Elektronik (aus der Aufbauanleitung).

Beide Motoren einer Seite sind durch je einen Motortreiber geregelt, wobei sich beide Motortreiber das Standby- sowie PWM-Signal teilen. Dadurch drehen alle Motoren stets betragsmäßig gleich schnell. In der Folge sind nur Geradeausfahrt vorwärts/rückwärts als auch bei entgegengesetzter Drehrichtung Kurven auf der Stelle möglich. Diese Vereinfachung kann für eine zukünftige Ausbaustufe eventuell überdacht werden, da die Positionserkennung durch Odometrie(Zählen der Rad-Rotationen) bei Rotation auf der Stelle verhätnismßig ungenau ausfällt. Dies ergab sich in den Inbetriebnahmetests und wurde durch das Intertialsystem kompensiert.

Hardwareabstraktion

Die zentrale Schnittstelle ist die abstrakte C++-Klasse `RobotHAL`, die in der Simulation parallel zur Entwicklung der Elektronik definiert wurde. Sie beschreibt die Fähigkeiteiten des Roboters(Geschwindigkeit je Seite, Objektdistanz nach vorne, Kantensensor, Drehrate, Encoderwinkel, Zeit, Servosteuerung und LED), und referenziert weder die Webots Implementierung noch die eigentlichen Hardwareimplementierungen. Diese abstrakte Basisklasse wird dann in der `HardwareImplementationHAL` implementiert und bindet dann die Controller der einzelnen Komponenten ein. Diese Controller sind im Projekt

Source Code als Template Klassen für die Hardwareparameter auf dem Arduino Framework implementiert. In der Hardwareabstraktion werden dann alle Controller mit den entsprechenden Pins aus einer Projektkonfigurationsdatei instanziiert und kombiniert.

Alle Komponenten sind *non-blocking* implementiert. Wenn z.B. die Motorsteuerung einer Seite sich derzeit in Vorwärtsdrehung befindet, dann durch die Steuerung aber ein Richtungswechsel ausgelöst wird, so muss zunächst aktiv abgebremst werden und erst wenn dieser Bremsvorgang beendet ist darf der Motor wieder in Rückwärtsrichtung angesteuert werden. Solche zeitabhängigen Vorgänge können natürlich durch `delay`/`sleep`-Befehle gelöst werden, in dieser Zeit erfolgt aber auch keine Sensorauswertung oder weitere Parametrierungen. Der Non-blocking Ansatz implementiert in den Komponenten jeweils eine interne Statemachine, die nach variablen Zeitintervallen einen Zustandsübergang durchführt und die Ausgangssignale anpasst oder nur in fixen Zeitintervallen Sensorwerte wiederum ausliest. Alle Aktionen finden in der Anwendungslogik weiterhin seriell statt, aber in geplanten Intervallen in `step` oder `update` Methoden, die in jedem Hauptschleifenzyklus durchlaufen werden und meistens direkt zurückspringen (Early Exit).

Anpassungen während der Umsetzung

Der ursprüngliche Ansatz für das Positionierungssystem war eine Integration des Beschleunigungssensors um einen Geschwindigkeitswert zu erhalten, Da die TT Motoren eine solche Information schlicht nicht bereitstellen können. Dieser Geschwindigkeitswert sollte wiederum integriert werden um eine absolute Positionierung zu erhalten.

Während der Hardwaretests zeigte sich, dass trotz Bias-Kalibrierung, Lagekorrektur und Stillstandserkennung in Verbindung mit der Motorsteuerung die erreichbare Genauigkeit für ein Positionssystem außerhalb akzeptabler Toleranz lag: Durch Testfahrten immer wieder vorwärts und rückwärts summierte sich ein Fehler auf die aktuelle Position von einigen Dezimetern mit steigender Tendenz. In der Folge wurde die Motorhalterung modifiziert und an der dem Rad abgewandten Seite zweier TT Motoren eine Gabellichtschranke mit 20-Loch Drehscheibe und LM393 Komparator angebracht. Durch diese Kodierung ist die Änderung des Radwinkels mit $1/40$ des Umfangs bestimmbar, bei Raddurchmesser von 66 Millimeter ergibt sich bei einem *Tick* am Sensor(Wert-Wechsel) eine Fahrtstrecke von $66 \times \pi/40 = 5.2$ Millimeter. Dies ist für eine gemessene Strecke zwar noch akzeptabel, aber bei Rotation wiederum zu ungenau um einen echten Drehwinkel für ein Navigationssystem zu bestimmen, weshalb der Drehwinkel weiterhin durch das IMU-System bezogen wird, wo Rotationswinkel weniger fehlerbehaftet sind als lineare Beschleunigungen. Bei der Implementierung der Tick-Erkennung wurde zusätzlich noch mit einem Hardwareinterrupt gearbeitet und konsekutive Ticks aufsummiert um sicherzustellen, dass nicht durch zu geringe Hauptschleifenfrequenz Ticks verloren gehen. Zudem wurden beide dieser Odometriecontroller mit der Motorsteuerung gekoppelt, sodass die Drehrichtung des jeweiligen Rades berücksichtigt wurde.

Weitergehend wurde festgestellt, dass eine Drehbewegung auf der Stelle ein deutlich höheres Drehmoment im Gegensatz zu Geradeausfahrt bedarf. Dies wurde in der Motorsteuerung berücksichtigt, allerdings ist der Schlupf der Räder auf einer Tischoberfläche hier sehr hoch. Vermutlich ist also in der Folge eine Navigation mit *sanften Kurven* doch zweckmäßiger. Dies sollte für eine weitere Ausbaustufe wie bereits erwähnt in Betracht gezogen werden.

Derzeit noch nicht umgesetzt sind die Akkuüberwachung (`HW-PWR-002/003`) sowie eine Rückkopplung von Odometriesystem zu Motorsteuerung um eine tatsächliche Geschwindigkeitsregelung auf $\frac{m}{s}$ zu erreichen. Die Ansteuerung der Motoren erfolgt über das PWM Signal, das lediglich von 0 für minimalem bis 255 für maximales Drehmoment parametriert werden kann, die tatsächliche Drehgeschwindigkeit ist aber dann wiederum von der Motorspannung `VM` abhängig.

1.5 Projektergebnis (Dennis Borutta)

Inhalt ca 0,5 Seiten

1.6 Zeitplan Soll/Ist (Dennis Borutta)

Inhalt ca 1 Seite

1.7 Kosten- und Ressourcenplan Soll/Ist (Gereon Such)

Inhalt ca 1 Seite

1.8 Nutzen und Weiterentwicklung des Ergebnis (Stefan Etringer)

Erreichter Stand

Der in Abschnitt 1.3 beschriebene Navigationsaufbau (A*, D* Lite, DWA, PoseEstimator) ist in der Simulation funktional vollständig und gegen die Testfälle `ST-SIM-001` bis `ST-SIM-006` verifiziert, ebenso die darüberliegende Sicherheitsebene (`SafetyMonitor`, `FaultMonitor`). Parallel dazu wurde die in der Softwaredokumentation zunächst nur als Vorschlag skizzierte Missionsablaufsteuerung (Tisch einlernen, Untersetzer ausgeben/einsammeln, Not-Halt) in einem eigenen Simulationsstand (`coasterbot_V_01_00`) umgesetzt und erreicht dort inklusive Hindernisumfahrung eine Positioniergenauigkeit von rund 1.5 cm. Auf der realen Hardware läuft bereits ein eigenständiger, direkt gegen die reale HAL arbeitender Punkt-zu-Punkt-Navigator (`Navigation`), der die Skid-Steer-Einschränkung (nur Geradeausfahrt und Drehung auf der Stelle, siehe Abschnitt 1.4)

berücksichtigt. Eine direkte Übernahme des simulierten A*/D*-Lite/DWA-Stapels auf den ressourcenbeschränkten Pico 2 steht noch aus.

Nutzen und Weiterentwicklung

Bevor die algorithmischen Entscheidungen auf der realen Hardware ausgeführt wurden, konnten diese simulationsgestützt verifiziert werden. Dies ist sichtbar am Beispiel der Odometrie: Die erste reale Iteration der Firmware verzichtete, wie ursprünglich geplant, auf Radencoder und integrierte die Position allein aus dem Beschleunigungssensor. Im Hardwaretest erwies sich die so erreichbare Genauigkeit als unzureichend (akkumulierter Fehler im Dezimeterbereich). Da der in der Simulation bereits erprobte `PoseEstimator` (Komplementärfilter aus Radodometrie und Gyroskop, verifiziert auf wenige Zentimeter bis Millimeter Abweichung bei Geradeausfahrt) zeigte, dass eine encoderbasierte Lösung zuverlässig funktioniert. Deshalb wurden nachträglich Gabellichtschranken an zwei Motoren ergänzt (siehe Abschnitt 1.4). Hierbei hat ein simulationsinterner Ansatz direkt eine Hardwareänderung begründet.

Den physischen Rahmen dafür liefern die Ergebnisse der Elektronik- und Mechanikentwicklung (Abschnitte 1.2 und 1.4): Ohne die aufgebaute Elektronik (Recheneinheit, Motortreiber, Sensorik und Energieversorgung) als reale Plattform zur Verifikation, wären die nötigen Änderungen an der Odometrie, Navigation oder Fehlererkennung nicht sichtbar geworden. Besonderen Nutzen für die Weiterentwicklung bringt dabei die eigens konstruierte Untersetzer-Ausgabe-/Aufnahmemechanik: Statt eines der generisch betrachteten Prinzipien (Sauger, Greifer, manuelle Einlage) wurde eine kombinierte Lösung aus magnetbestückter Hubscheibe und Schubkurbeltrieb entwickelt, ergänzt um einen 90°-indexiert drehenden Ausschieber, der den Untersetzer beim Ablegen ausgerichtet auf einen Führungssteg schiebt und beim Aufnehmen den Weg für den Heber freigibt. Diese Mechanik wird über einen einzigen Servomotor angesteuert und besitzt damit bewusst nur einen Freiheitsgrad. Deshalb genügt zur Steuerung eine einfache servogesteuerte Ablaufprozedur, und der in der Alternativenbeschreibung noch mit inverser Kinematik und RRT* skizzierte, deutlich aufwendigere Ansatz (Abschnitt 1.3) war für die gewählte mechanische Lösung nie erforderlich.

Bei der Missionsablaufsteuerung bildet die Simulation inzwischen nicht mehr jeden Aspekt ab, der bereits Teil der realen Software ist: Im hier dokumentierten Hauptsimulationsstand existiert sie nur als Vorschlag, während sie im parallelen Stand `coasterbot_V_01_00` sowie in der Firmware bereits aktiv weiterentwickelt wird.

Für die Weiterentwicklung des Projekts ergeben sich nach Möglichkeit daraus folgende nächste Schritte: die Zusammenführung der beiden Simulationsstände zu einem gemeinsamen Navigations- und Missionsmodell, sowie die schrittweise Übertragung des verifizierten Navigationssaufbaus auf die reale Firmware Zunächst kann der bereits erfolgreich erprobte `PoseEstimator`-Ansatz, danach die D*-Lite-/DWA-basierte Hindernisvermeidung, eingesetzt werden insofern ausreichend Rechenzeit auf dem Pico 2 zur Verfügung steht.

2 Projektablauf

Planumfang ca. 9 Seiten Davon: Dennis 3 Seiten, Stefan 3 Seiten, Gereon 3 Seiten

2.1 Teambildung und Rollenverteilung (Dennis Borutta)

ca 1 Seite

- Wie war die Teambildung für das Projekt?
  - Kurzes objektives resüme aus dem Projektstart
- Wie fand die Rollenverteilung statt?
  - Kurzes objektives resüme aus dem Projektstart

2.2 Zusammenarbeit im Projekt (Stefan Etringer)

Organisatorischer Rahmen

Das Projekt ist in die Meilensteine M0, M3, M4 und M5 gegliedert (Recherche, Produktentwicklung, Umsetzung, Test und Validierung, Endbericht), denen im Repository je ein eigenes Verzeichnis entspricht. Innerhalb des zeitlichen Rahmens wurde die Arbeit unter den drei Teammitgliedern nach technischen Gewerken aufgeteilt: Dennis Borutta verantwortete den mechanischen Aufbau, Gereon Such die Elektronik und die Hardwareabstraktion, Stefan Etringer die Navigationsalgorithmik und deren Simulation. Diese Aufteilung blieb über die Meilensteine hinweg weitestgehend gleicht. Die Zuordnung einzelner Arbeitspakete und Termine erfolgte über Notion (siehe Abschnitt 2.3).

Zusammenarbeit über die Versionsverwaltung

Technisch wurde über ein gemeinsames Git-Repository auf GitHub zusammengearbeitet (siehe Abschnitt 2.3). Jedes Teammitglied committete überwiegend direkt auf den `main` Branch. Größere, zusammenhängende Änderungen legte sich jedes Mitglied bei Bedarf einen eigenen Arbeits-Branch an (`Dennis_Workbranch`, `Stefan_Workbranch`), der anschließend auf `main` zusammengeführt wurde. Ein formaler Review-Prozess über Pull Requests fand dabei nicht statt. Bei einem Team dieser Größe wurden Rückfragen zu Änderungen direkt in den wöchentlichen Besprechungen über Alfaview oder über WhatsApp geklärt (siehe erneut Abschnitt 2.3).

Die schriftlichen Dokumente (Lastenheft, Pflichtenheft, Softwaredokumentation, Testkonzept, dieser Endbericht, usw.) wurden nach zuvor festgelegtem Template gemeinsam in LaTeX verfasst. Änderungen an den Dokumenten wurden in der Tabelle der Versionshistorie festgehalten.

Schnittstellen zwischen den Gewerken

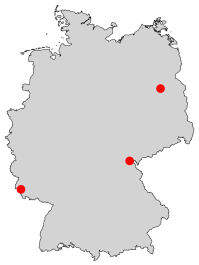
Die genannten Verantwortungsbereiche greifen im letztendlichen Hardwareaufbau eng ineinander. Um eine zeitgleiche Arbeit an den Kompontenten zu ermöglichen, wurde eine abstrahierende Schnittstelle zur losen Kopplung zwischen diesen eingeführt. Zentral dafür ist die Hardwareabstraktionsschicht `RobotHAL` (Abschnitt 1.4): Da die Navigations- und Sicherheitslogik gegen dieses Interface entwickelt wird, konnte die Algorithmik (Abschnitt 1.3) in der Simulation geprüft werden, während parallel die reale Hardwareanbindung aufgebaut wurde, ohne dass eine Seite auf den jeweils aktuellen Stand der anderen warten musste. Entkoppelt wurde die Mechanik (Abschnitt 1.2) über die Ansteuerung einzelner Servo-Freiheitsgrade angebunden. Eine engere Abstimmung war dort nötig, wo sich diese Schnittstellen im Projektverlauf änderten, etwa bei der nachträglichen Ergänzung der Radencoder (Abschnitt 1.8). Hierbei wurde die Erweiterung der `RobotHAL` zwischen Algorithmik und Elektronik abgestimmt, bevor sie umgesetzt wurde.

2.3 Kommunikation im Projekt (Gereon Such)

Kommunikation Gereon Such: ca 1 Seite

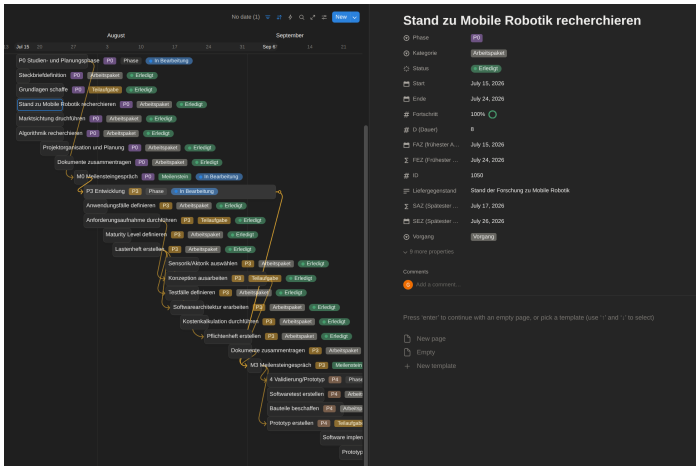
- Darstellung und Bewertung der Kommunikation im Projekt
- Objektive Beschreibung. Bewertung später.
- WhatsApp, Alfaview, Notion
- Darstellung und Bewertung der aufgetretenen Probleme bzgl. Teamarbeit und Kommunikation; Lösung dieser Probleme
- Objektive Beschreibung. Bewertung später.

Das Coasterbot Team ist bei Stichprobengröße $N = 3$ in guter Näherung über ganz Deutschland verteilt und die Kommunikation fand folglich komplett digital statt. Die Teamleitung und Kommunikation in den Meilensteingesprächen wurde durch Dennis Borutta wahrgenommen. Die Aufgaben wurden in Arbeitspakete auf- und im Team verteilt. Ein wöchentliches Meeting wurde per Videokonferenz durchgeführt, in denen jedes Teammitglied Fortschritte mitgeteilt, auf eventuelle Probleme hingewiesen sowie um eventuelle Hilfe gebeten hat. Kurze Rückmeldungen dazwischen wurden in der Woche über eine WhatsApp-Gruppe geteilt. In der Kommunikation traten keine Probleme auf.



Projektverwaltung: Notion

Das Zentrale Projektsteuerungstool war **Notion**[1]. Dieses wurde zur Bestimmung von Verantwortlichkeiten, Unteraufgaben, Deadlines und Terminen genutzt, hauptsächlich gepflegt durch Dennis Borutta. Es ermöglicht neben einer wie der rechts dargestellten Timelinedarstellung auch ein kollaboratives Dokumentenmanagement, welches durch das Team insbesondere in der Anfangsphase des Projekts für Recherche und Brainstorming genutzt wurde.



Kollaborationsplatform: Github

Das Projekt inklusive aller in LaTeX erstellten Dokumente, Sourcecode und anderer Entwicklungsdateien sind in einem Github[2] Projekt verfügbar[3]. Git[4] ist ein Versionsverwaltungssystem für das dezentrale Verwalten verschiedener Versionsstände und ist für das Coasterbot Projekt gewählt worden weil das Team bereits vertraut mit diesem Tool war, es kostenlos und Cross-Platform verfügbar ist. Github ist wiederum die zentrale Platform, an dem die Versionsstände wiederum gehostet werden.

Videokonferenz: Alfaview

Als Videokonferenzsystem für den wöchentlichen Austausch wurde auf eine Lösung der deutschen Firma Alfaview[5] aus Karlsruhe gesetzt. Es ermöglicht neben regulärer Videokonferenz sowohl das Teilen von Dateien als auch ein Screensharing, was für den Austausch der Arbeitsstände enorm vorteilhaft war.

Kommunikation: WhatsApp

Für schnelle informelle Abstimmung und Terminabsprachen wurde der Instant-Messanging WhatsApp[6] genutzt. Die Kommunikation erfolgte hier aufgrund der unterschiedlichen Berufsauslastung und familärer Situation im Team asynchron.

[1] https://www.notion.com
[2] https://github.com
[3] https://github.com/Project-CoasterBot/coasterbot
[4] https://git-scm.com
[5] https://alfaview.com
[6] https://whatsapp.com

2.4 Bewertung der Zusammenarbeit

ca 3 Seiten

- Darstellung und Bewertung der Zusammenarbeit im Projektteam
  - Subjektives Empfinden.
- Darstellung und Bewertung der Kommunikation im Projekt
  - Subjektives Empfinden.
- Darstellung und Bewertung der aufgetretenen Probleme bzgl. Teamarbeit und Kommunikation; Lösung dieser Probleme
  - Subjektives Empfinden.

Idee: Vielleicht gibt erstellen wir eine Miniumfrage und zeigen hier die Ergebnisse.

Restrospektive Gereon Such:

ca 1 Seite

Restrospektive Stefan Etringer:

ca 1 Seite

Restrospektive Dennis Borutta:

ca 1 Seite

2.5 Bewertung des Projektergebnis

Inhalt ca 3 Seiten

- Subjektives Empfinden. Was lief gut, was würden wir besser machen.

Perspektive Gereon Such:

ca 1 Seite

Die Bewertung des Ergebnisses möchte ich gerne differenzieren. So bin ich mit meiner persönlichen Lernkurve sowie dem Team vollumfänglich zufrieden. Ich habe einigermaßen Erfahrung im Embedded-Bereich, sowohl auf Hard- als auch Softwareebene, aber mit Aktorik und Energiemanagement habe ich wenig Erfahrung. Hier habe ich wiederum viel dazugelernt, muss aber eingestehen, dass ich mit meinem persönlichen Projektbeitrag nicht gänzlich zufrieden bin.

Da ich die Verantwortung für Konzeption und Entwicklung der Elektronik sowie das Abstrahieren dieser Funktionalitäten auf der Microcontrollerplattform übernommen habe sehe ich mich auch verantwortlich für Probleme der finalen Plattform, die daraus

resultieren. Die meisten der getroffenen Entscheidungen und deren Umsetzung haben sich bewährt, aber insbesondere mit der Bewegungssteuerung und damit verbundener Navigation bin ich nicht zufrieden.

- DC-Motoren ziehen insbesondere beim Anfahren als auch bei stall(blockieren) erhebliche Ströme, eine Energiereglung (Linearregler, Buck Converter etc.), die diese Ströme liefern kann, ist teuer. Daher habe ich die Energieversorgung mit Zwei Pegeln konzipiert: Einer Motorspannung direkt am Akku der hohe Ströme liefern kann sowie einem VCC Pegel von 5V für die elektronischen Komponenten, die durch einen Regler bereitgestellt wird. Die Motoren haben einen Spannungsbereich von 3 bis 6 V, der gewählte Akku hat 3.7V, ist also näher am unteren Ende des Eingangsspannungsbereichs, möglicherweise wären 4 AA Batterien eine bessere Entscheidung gewesen.
- Das notwendige Anfahrdrehmoment auf den Rädern ist bei einer Drehung auf der Stelle erheblich höher als bei Geradeausfahrt. Vor der eigentlichen Festlegung auf die Komponenten habe ich die TT Motoren auf einem sehr leichten Rahmen getestet, hier waren die Motoren für Drehungen auf der Stelle völlig unproblematisch. Die Coasterbot Plattform ist durch die Dimensionierung und den Coastermechanismus allerdings bedeutend schwerer und hier ist das mögliche Drehmoment der Motoren grenzwertig. Bei Drehbewegungen auf der Stelle gibt es erheblichen Schlupf zwischen Antriebsrad und der Tischplatte, wodurch auch eine Orientierung per Odometrie nicht mehr möglich ist. Zum Zeitpunkt der Komponentenauswahl war zwar das Gewicht der Plattform nicht bekannt, dennoch würden stärkere Motoren sicherlich weniger Probleme verursachen. Ebenso wäre eine Designentscheidung mit Kurvenfahrt statt Drehung auf der Stelle hier möglicherweise eine Lösung, dies wurde aber durch die Designentscheidung von kombinierter Ansteuerung von linker und rechter Seite verhindert.
- Durch die innenseitige Montage der Motoren und Achsführung nach Außen sitzt das Rad nur teilweise auf der Achse, da es sonst am Gehäuse schleift und ist dadurch tendenziell instabil.
- Das ursprünglich gewählte Inertialsystem durch Integration auf dem Gyroskop Sensor MPU-6050 war in der Praxis nicht praktikabel und ich musste eine Odometrie Zählung an den Achsen ergänzen. Hier gibt es DC-Motoren, die solche Zähler bereits eingebaut haben.

Von diesen Problemen abgesehen ist das Gesamtprojekt als Proof-of-Concept einer Robotikplatform aus meiner Bewertung dennoch ein Erfolg und ich bin sehr froh, dass ich mich beteiligen und mit Dennis Borutta und Stefan Etringer zusammenarbeiten durfte.

Eine mögliche Lösung für das Positionssystem wäre auf dieses komplett zu verzichten und stattdessen eine optische Erkennung der Umgebung mit der ohnehin geplanten Kamera zu implementieren. Ein solches System kann Ziele und Geometrie der überschaubaren

Oberfläche eines Tisches vermutlich hinreichend erfassen statt auf ein blindes Positionierungssystem angewiesen zu sein. Dies wäre eine mögliche Aufgabe für ein zukünftiges Projekt.

Perspektive Stefan Etringer:

ca 1 Seite

Perspektive Dennis Borutta:

ca 1 Seite